

Algebraic Data Types in Isabelle

Lecture Notes Transcription

Course: Interactive Theorem Proving (7CCSACTP)

1. Context & Intuition

Revision of Previous Weeks:

- Isabelle Interface & Theorem Stating.
- Proof techniques: FW, BW, **subst**, induction on **nat**.
- Structural recursion & structural induction on lists.

Datatype Intuition:

- A datatype definition formally encapsulates:
 1. What are the possible constructors / patterns.
 2. How recursive calls on that datatype progress.
- **Lists:** $x_1 \# x_2 \# \dots \# x_n \# [] \implies$ Patterns: **Nil** (`[]`) and **Cons** `x xs` ($x \# xs$).
- **Natural numbers:** Patterns: 0 and **Suc** n .

2. Formal Datatype Definitions in Isabelle

Lists ('a list):

```
datatype 'a list = Nil (Constructor 1)
                | Cons 'a "'a list" (Constructor 2)
```

Natural Numbers (nat):

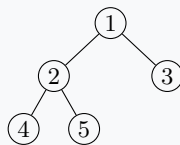
```
datatype nat = 0 (Constructor 1)
             | Suc "nat" (Constructor 2)
```

3. Binary Trees

Node-labeled Binary Trees:

```
datatype 'a tree = Leaf | Node "'a tree" 'a
                  "'a tree"
```

Example Tree Structure:



Term Representation:

```
Node (Node (Node Leaf 4 Leaf) 2 (Node Leaf 5
Leaf)) 1 (Node Leaf 3 Leaf)
```

Leaf-labeled Binary Trees (Alternative):

```
datatype 'a tree = Leaf 'a | Node "'a tree"
                  "'a tree"
```

Structure: Values are stored exclusively at the leaves, and internal nodes only join subtrees together.